
Universidade Estadual do Piauí – UESPI
Campus de Parnaíba
Tecnologia em Sistemas de Computação

Introdução a Computação

Aula 09 – Armazenamento de Dados
3 - Números Reais

Prof. Francisco Rocha

1

Armazenamento de Dados

• Percurso

– Representação e Armazenamento de Dados

- 1 - Introdução
- 2 - Números Inteiros
- **3 - Número Reais**
- 4 - Texto
- 5 - Imagem, Áudio e Vídeo

2

Armazenamento de Dados

• Armazenamento de Números

- Armazenamento do ponto decimal
 - Ponto fixo
 - Números inteiros
 - Número sem uma parte fracionária
 - Ponto flutuante
 - Números reais
 - Números com uma parte fracionária

3

Armazenamento de Dados

• Armazenamento de Números

- O tamanho do padrão binário irá definir o intervalo de valores que conseguirmos representar.
- Por exemplo:
 - Com 5 bits representa-se números inteiros com sinal que compreende os valores de -16 até 15.
 - Com 5 bits representa-se números inteiros sem sinal que compreende os valores de 0 até 31.
 - Com 8 bits (1 byte) representa-se números inteiros com sinal que compreende os valores de -128 até 127.
 - Com 8 bits (1 byte) representa-se números inteiros sem sinal que compreende os valores de 0 até 255.

4

Armazenamento de Dados

- **Armazenamento de Números**

- Em linguagens de programação de tipagem estática, usando o tipo de dados byte para declarar variáveis:

```
byte a;  
a = 100;  
exibir("Valor de a: " + a);    // Valor de a: 100  
a = 127;  
exibir("Valor de a: " + a);    // Valor de a: 127  
a = 128;  
exibir("Valor de a: " + a);    // Valor de a: -128  
a = 200;  
exibir("Valor de a: " + a);    // Valor de a: -56
```

5

Armazenamento de Dados

- **Armazenamento de Números Reais**

- Como armazena-se um número real no padrão binário?

- Exemplos de números reais

127,56

-127,56

123465321654651,5

65465134165,2465465416

-2,23165465416

0,53643

6

Armazenamento de Dados

• Armazenamento de Números Reais

- Exemplos de números reais:



- Quantos bits reservar para armazenar a parte inteira e para a parte fracionária? Não convém separar as partes inteira e parte fracionária no padrão binário.

+123465321654651,5

+65465134165,2465465416

-2,23165465416

7

Armazenamento de Dados

• Armazenamento de Números Reais

- A estratégia adotada será baseada no "ponto flutuante" para que o "ponto" flutue.
- Um valor binário de ponto flutuante é definido pela seguinte fórmula:

$$\text{sinal} * \text{mantissa} * \text{base}^{\text{exp}}$$

- Essa representação permite variar a quantidade necessária para representar a parte inteira e fracionária.
- Essa representação será facilitada utilizando uma representação de ponto fixo.

8

Armazenamento de Dados

• Armazenamento de Números Reais

- Exemplos de números na representação de ponto flutuante, formado como um número de ponto fixo:

a) 123456789.8765

1.234567898765 x 10⁸ (notação científica)

1.234567898765E8 (notação científica)

b) 0.00000123456789

1.23456789 x 10⁻⁶ (notação científica)

1.23456789E-6 (notação científica)

- A notação científica é a representação utilizada para armazenar números reais no padrão binário.

9

Armazenamento de Dados

• Armazenamento de Números Reais

- Exemplos de números na representação de ponto flutuante, formado como um número de ponto fixo:

+123456789.00

+1.23456792E8 (notação científica)

+1.23456792 x 10⁸

Sinal

Expoente

Número

Antes, será preciso
converter o número
para binário.

10

Armazenamento de Dados

• Converter Parte Fracionária para Binário

- Para converter a parte fracionária de um número decimal em binário, comece com a fração em questão e multiplique-a por 2 observando o inteiro resultante e a parte fracionária. Continue multiplicando por 2 até obter uma parte fracionária resultante igual a zero. Em seguida, escreva as partes inteiras dos resultados de cada multiplicação.

- Exemplo de parte fracionária: 0.375
- $$\begin{aligned}0.375 * 2 &= 0 + 0.75 \\0.75 * 2 &= 1 + 0.5 \\0.5 * 2 &= 1 + 0\end{aligned}$$

- Logo, 0.375 em binário equivale a 0.011

11

Armazenamento de Dados

• Sistema Binário

- No sistema binário, um número pode ser escrito como

$$(S_{K-1} \dots S_2 S_1 S_0 . S_{-1} S_{-2} S_{-3} \dots S_{-L})_2$$

- tem o valor de:

$$n = \pm \underbrace{S_{K-1} \times 2^{K-1} + \dots + S_1 \times 2^1 + S_0 \times 2^0}_{\text{Parte Inteira}} + \underbrace{S_{-1} \times 2^{-1} + S_{-2} \times 2^{-2} + \dots + S_{-L} \times 2^{-L}}_{\text{Parte Fracionária}}$$

12

Armazenamento de Dados

- Sistema Binário

– Converter $(0.011)_2$ para 0.375_{10} :

$$(S_{-1} \times 2^{-1} + S_{-2} \times 2^{-2} + S_{-3} \times 2^{-3} + \dots + S_{-L} \times 2^{-L})$$

$$011 =$$

$$0 \times 2^{-1} + 1 \times 2^{-2} + 1 \times 2^{-2} =$$

$$0 \times 0.5 + 1 \times 0.25 + 1 \times 0.125 =$$

$$0 + 0.25 + 0.125 =$$

$$0.375$$

13

Armazenamento de Dados

- Sistema Binário

– Converter $(0.111111)_2$ para $(?)_{10}$:

$$(S_{-1} \times 2^{-1} + S_{-2} \times 2^{-2} + S_{-3} \times 2^{-3} + \dots + S_{-L} \times 2^{-L})$$

$$111111 =$$

$$1 \times 2^{-1} + 1 \times 2^{-2} + 1 \times 2^{-3} + 1 \times 2^{-4} + 1 \times 2^{-5} + 1 \times 2^{-6} =$$

$$1 \times 0.5 + 1 \times 0.25 + 1 \times 0.125 + 1 \times 0.0625 + 1 \times 0.03125 + 1 \times 0.015625 =$$

$$0.5 + 0.25 + 0.125 + 0.0625 + 0.03125 + 0.015625 =$$

$$0.984375$$

14

Armazenamento de Dados

• Sistema Binário

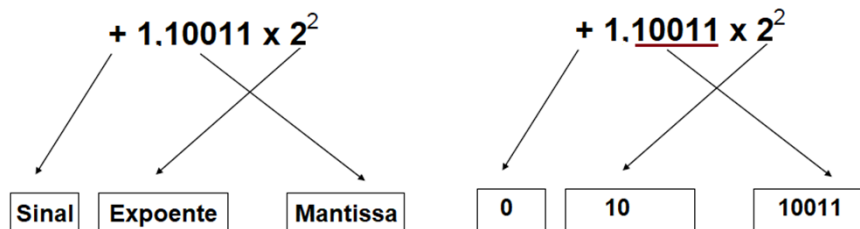
- Por exemplo, o número $(6.375)_{10}$ seria representado em binário por $(110.011)_2$.
- A parte inteira '6' é representada por 110.
- A parte fracionária '0.375' é representada por 011.
- Em seguida, o número 110.011 será representado pela notação de ponto fixo para então ser armazenado.
- Assim,
 - Decimal: +6.375
 - Binário: +110.011
 - Notação Científica: $+1.10011 \times 2^2$ ou $+1.10011E2$

15

Armazenamento de Dados

• Armazenamento de Números Reais

- O número,
 - Decimal: +6.375
 - Binário: +110.011
 - Notação Científica: $+1.10011 \times 2^2$ ou $+1.10011E2$



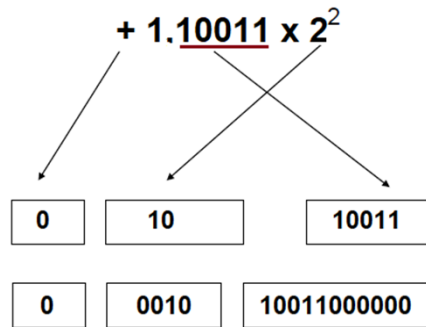
- O dígito '1' à esquerda está implícito e não é armazenado.

16

Armazenamento de Dados

• Armazenamento de Números Reais

- Precisa-se definir a quantidade de bits que serão utilizados para o expoente e para a mantissa.



Ao considerar 16 bits, temos:

Sinal: 1 bit

Expoente: 4 bits

Mantissa: 11 bits

- Assim, temos o número real: **0001010011000000**

17

Armazenamento de Dados

• Armazenamento de Números Reais

- Recuperando o valor armazenado no padrão binário:

0001 0101 0100 0000.

- Considerando 16 bits:

- Sinal (1 bit): **0**
- Expoente (4 bits): **0010**
- Mantissa (11 bits): **10101000000**

- Recupera-se o '1' implícito: **1.10101000000**

- Desloca-se a vírgula: **110.101000000**

- Converte a parte inteira: **$(110)_2 = (6)_{10}$**

- Converte a parte fracionária: **$(0.101000000)_2 = (0.625)_{10}$**

- Concatena e adiciona o sinal: **$(0110101000000)_2 = (+6.625)_{10}$**

18

Armazenamento de Dados

• Armazenamento de Números Reais

- O expoente pode ser positivo ou negativo.
 - Poderia ser representado com a representação complemento de dois.
- O expoente será representado de uma maneira que se armazene somente inteiros sem sinal (inteiros positivos).
- Para isso, um inteiro (chamado **bias**) é adicionado ao valor do expoente.

19

Armazenamento de Dados

• Armazenamento de Números Reais

- O bias do expoente é calculado como:

$$2^{m-1} - 1$$

- **m** representa o número de bits reservado para armazenar o expoente.
- Com 4 bits o bias seria: $2^{m-1} - 1 = 2^{4-1} - 1 = 2^3 - 1 = 8 - 1 = 7$
- Esse sistema de representação do expoente é chamado de Excesso.
- No caso, temos o Excesso 7.
 - Pois sempre se soma 7 ao valor do expoente antes de armazená-lo como binário.

20

Armazenamento de Dados

• Armazenamento de Números Reais

- Na representação de excesso 7: Soma 7 ao valor do expoente.
- Exemplo para: **+110.101**
 - Normalização: **1.10101** $\times 2^2$
 - Sinal: **0**
 - Expoente: $(2)_{10} = (10)_2$, em excesso 7 é $(2+7) = (9) = (1001)_2$
 - Mantissa: **10101**
 - Assim, o padrão binário fica (em 16 bits): **0100110101000000**
 - O expoente é um número inteiro sem sinal em excesso.

21

Armazenamento de Dados

• Armazenamento de Números Reais

- Recuperando um número real armazenado no sistema de representação de excesso 7.
 - Subtrai 7 ao valor do expoente.
- Exemplo para 0100 1101 0100 0000 :
 - Padrão binário: **0100 1101 0100 0000**
 - Sinal: **0**
 - Expoente: **1001 = 9 – 7 = 2**
 - Mantissa: **10101000000**
 - Número recuperado: **1.10101000000** ou **110.101000000**
 - Ou ainda **+110.101000000 = (+6.625)₁₀**

22

Armazenamento de Dados

• Armazenamento de Números Reais

- Na representação de excesso 7: Soma 7 ao valor do expoente.
- Exemplo para $(-0.75)_{10} = (-0.11)_2$
 - Normalização: -1.1×2^{-1}
 - Sinal: 1
 - Expoente: $-1+7 = 6 = 0110$
 - Mantissa: 1
 - Padrão binário: **1011 0100 0000 0000**

23

Armazenamento de Dados

• Armazenamento de Números Reais

- Recuperando um número real armazenado no sistema de representação de excesso 7.
 - Subtrai 7 ao valor do expoente.
- Exemplo para 1011 0100 0000 0000
 - Padrão binário: **1011 0100 0000 0000**
 - Sinal: 1
 - Expoente: $0110 = 6 - 7 = -1$
 - Mantissa: **1000000000**
 - Número recuperado: **1.1000000000E-1** ou **0.110000000000**
 - Ou ainda **-0.110000000000 = $(-0.75)_{10}$**

24

Armazenamento de Dados

• Erro de Truncamento

- O erro de truncamento ocorre quando a quantidade de bits não é suficiente para armazenar o valor, por exemplo:
 - Considere 11 bits para armazenamento da mantissa
 - Número binário: 1111111111.111111
 - Normalização...: $1.\underline{111111111111111} \times 2^9$
 - A mantissa possui 15 bits: 111111111111111
 - Somente 11 serão armazenados: 11111111111111
 - Os bits mais à direita serão excluídos
 - Valor recuperado: 1.11111111111 ou 1111111111.11

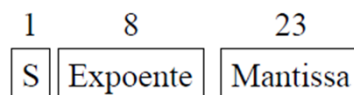
25

Armazenamento de Dados

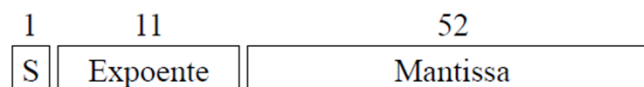
• Armazenamento de Números Reais

- Exemplo de padrões para representação de ponto flutuante:

- Excesso 127 – Precisão simples (32 bits)



- Excesso 1023 – Precisão dupla (64 bits)



26

Armazenamento de Dados

• Exemplo em Java

```
System.out.println("Float: "+Float.SIZE+" bits");
System.out.println("Float - Max expoente: "+Float.MAX_EXPONENT);
System.out.println("Float - Min expoente: "+Float.MIN_EXPONENT);

System.out.println("Double: "+Double.SIZE+" bits");
System.out.println("Double - Max expoente: "+Double.MAX_EXPONENT);
System.out.println("Double - Min expoente: "+Double.MIN_EXPONENT);
```

- Float: 32 bits
- Float - Max expoente: 127
- Float - Min expoente: -126
- Double: 64 bits
- Double - Max expoente: 1023
- Double - Min expoente: -1022

27

Armazenamento de Dados

• Referências Bibliográficas

- Behrouz Forouzan e Firouz Mosharraf. Fundamentos da Ciência da Computação. 2ed. Cengage Learning, 2012.
- J. Gleen Brookshear. Ciência da Computação: uma visão abrangente. 11ed. Bookman, 2013.
- Fernando C. Veloso. Informática: conceitos básicos. 8.ed. Elsevier, 2011.
- Marcelo Marcula; Pio Armando Benini Filho. Informática: conceitos e aplicações. 4ed. Érica, 2013.
- Ricardo Daniel Fedeli; Enrico Giulio Franco Polloni; Fernando Eduardo Peres. Introdução à ciência da computação. 2ed. Cengage Learning, 2010.

28

Armazenamento de Dados

